

STUDENT TECHNICAL REPORT / SUBMISSION RECORD

Assignment 2 - Real-Time Data Processing with Pandas and NumPy

AITU Smart Classroom Energy Stream

This report records the implementation and evidence for the supplied synthetic classroom sensor stream. The calculations, plots, and test results referenced below are generated from the submitted project files.

1. Student and Submission Information

Required field	Student entry
Student full name	Amankos Dania Dauletuly
Student ID	240260
Group / cohort	BDA-2405
Instructor	Serek Azamat
Submission date	21.09.2026
Individual variant / task code	Not provided in the assignment file
Git repository / project link	Add after GitHub upload
Python / tool versions	Python 3.10.11; pandas 2.3.3; NumPy 2.3.3; Matplotlib 3.10.6; pytest 8.4.2

2. Work Record

Item	Student entry
Work start date	21 September 2026
Work completion date	21 September 2026
Main libraries / technologies	Python, Pandas, NumPy, Matplotlib, pytest
Source files / notebook	main.py
Test / experiment files	test_assignment2.py; evidence/calculation_check.json
Final repository commit/version	Add after GitHub upload

3. Report Structure

2.1 Data Quality and Cleaning

The raw sensor data contains three missing readings: temperature at 10:02, CO2 at 10:04, and occupancy at 10:07. I used the cleaning methods specified in the assignment. Temperature and CO2 are linearly interpolated, while occupancy is forward-filled.

Variable	Missing at	Method	Cleaned value	Reason
Temp_C	10:02	Linear interpolation	21.9	Between 21.7 and 22.1
CO2_ppm	10:04	Linear interpolation	750	Between 720 and 780 ppm
Occupancy	10:07	Forward fill	29	Most recent known occupancy
Required evidence		Student response / reference		
Technical explanation		Required cleaning methods are applied only to the three columns with missing sensor values.		
Code / file / function reference		main.py - missing_value_summary(), clean_data(), validate_cleaned_data()		
Result / calculation / experiment evidence		cleaned_classroom_data.csv; evidence/main_output.txt		
Student-specific decision or observation		After forward fill, Occupancy is converted back to integer because people are counted as whole values.		

2.2 Streaming Window Statistics

I used a 3-row rolling window for temperature, CO2, and power. Since the data arrives once per minute, each complete window represents three consecutive minutes. The calculation uses `min_periods=3`, so rolling values begin at 10:02 when the first complete three-reading window exists.

Time	Temp mean	CO2 mean	Power mean
10:02	21.700	653.333	3.367
10:03	21.900	686.667	3.567
10:04	22.100	720.000	3.767
10:05	22.300	750.000	4.000
10:06	22.467	780.000	4.200
Required evidence		Student response / reference	
Technical explanation		Each rolling value is the mean of the current row and previous two cleaned readings.	
Code / file / function reference		main.py - add_rolling_statistics()	
Result / calculation / experiment evidence		processed_classroom_data.csv; completed_tables.md; evidence/main_output.txt	
Student-specific decision or observation		I require all three readings before reporting a rolling value instead of producing partial-window means.	

2.3 Feature Engineering

Three required features are added after cleaning. Occupancy density measures the share of the 40-person room capacity in use. Energy per student normalizes power by occupancy. The comfort flag checks the assignment limits for CO2 and temperature.

Feature		Formula	
occupancy_density		Occupancy / 40	
energy_per_student		Power_kW / Occupancy	
comfort_flag		CO2_ppm <= 1000 and 20 <= Temp_C <= 24	
Time	Density	Energy/student	Comfort
10:03	0.600	0.1583	True
10:06	0.725	0.1517	True
10:09	0.825	0.1545	True
10:11	0.925	0.1541	True
Required evidence		Student response / reference	
Technical explanation		The features convert raw sensor values into measures that are easier to interpret operationally.	
Code / file / function reference		main.py - add_features()	
Result / calculation / experiment evidence		processed_classroom_data.csv; completed_tables.md	
Student-specific decision or observation		The comfort limits are inclusive, so exactly 24 C and 1000 ppm still count as comfortable.	

2.4 Statistical Analysis

The statistical summary is calculated from the cleaned data. The highest-power time is selected with `idxmax()`, and Pearson correlation is calculated between occupancy and power.

Metric	Result
Temperature mean	22.4833 C
Temperature median	22.55 C
Temperature minimum	21.5 C
Temperature maximum	23.3 C
CO2 mean	789.1667 ppm
CO2 maximum	950 ppm
Highest power	10:11 - 5.7 kW
Occupancy-power correlation	0.9946

The correlation is very strong and positive in this synthetic sample. Higher occupancy is associated with higher power consumption. This is an association, not proof that occupancy alone causes the increase.

Required evidence	Student response / reference
Technical explanation	Pandas summary operations are used for central tendency, range, maximum-power selection, and correlation.
Code / file / function reference	main.py - calculate_statistics()
Result / calculation / experiment evidence	evidence/main_output.txt; evidence/calculation_check.json

2.5 Visualization and Engineering Interpretation

The first plot shows the 3-minute rolling temperature and CO2 trends over time. The second plot compares occupancy and power, with a fitted line to make the relationship easier to see.

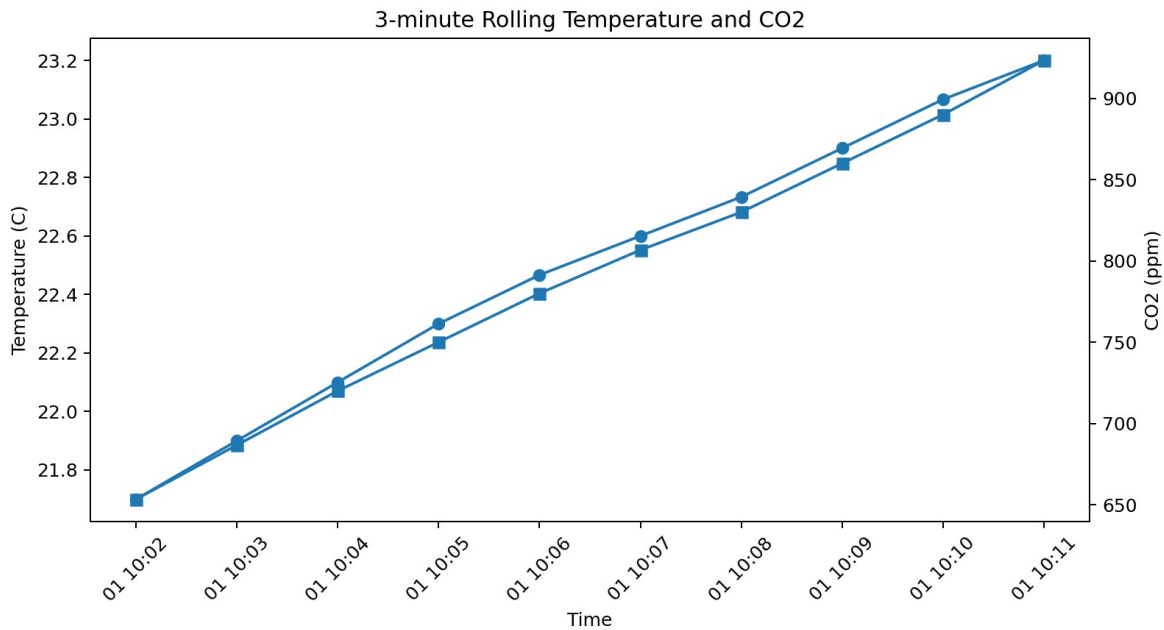


Figure 1. 3-minute rolling temperature and CO2.

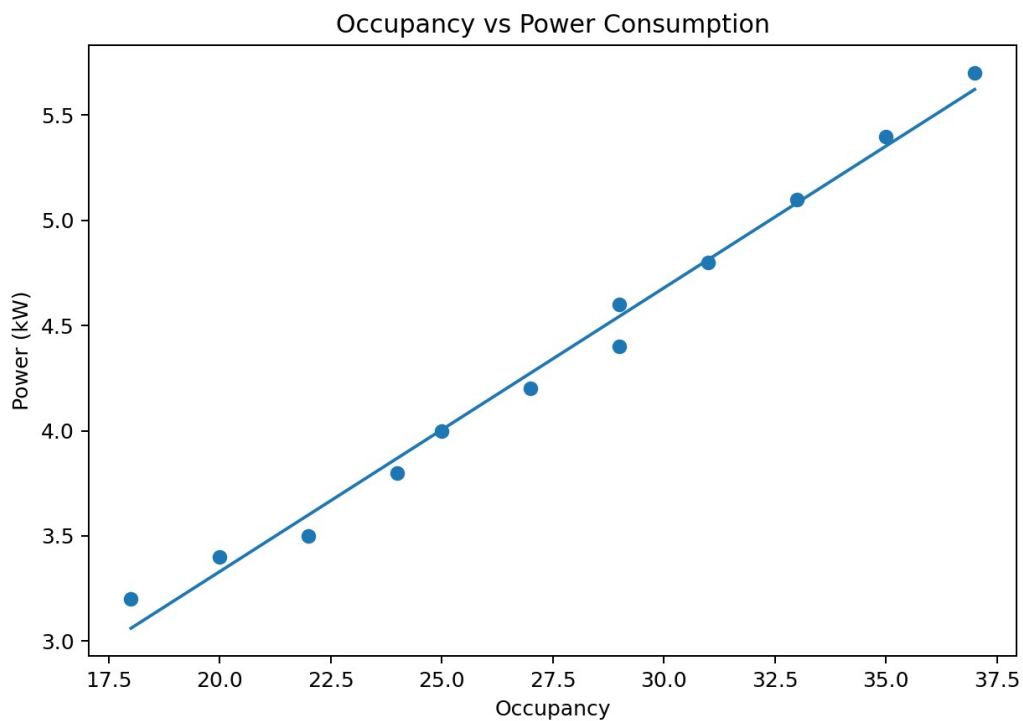


Figure 2. Occupancy versus power consumption.

Operational insight 1: Occupancy and power rise closely together. Occupancy-aware schedules or controls could help reduce energy use when a room is lightly occupied or empty.

Operational insight 2: Rolling temperature and CO2 rise through the class period. Facility staff could monitor the trend as an early signal to review ventilation or room-control settings.

Required evidence	Student response / reference
Technical explanation	The plots show time-based environmental change and the occupancy-power relationship.
Code / file / function reference	main.py - create_plots()
Result / calculation / experiment evidence	plots/rolling_temperature_co2.png; plots/occupancy_vs_power.png
Student-specific decision or observation	The rolling plot uses separate y-axes because temperature and CO2 have very different numerical scales.

2.6 Technical Reflection

A key processing decision was to use the exact missing-value methods specified in the task and to delay rolling statistics until a full three-reading window is available. One limitation is that the dataset contains only one room and twelve minutes, so the observed correlation should not be generalized to real AITU buildings. A realistic improvement would be to process multiple rooms over longer periods and compare patterns by room, time of day, and occupancy level.

Required evidence	Student response / reference
Technical explanation	Cleaning is completed before rolling and feature calculations so later results are based on a consistent dataset.
Code / file / function reference	main.py - main() processing order
Result / calculation / experiment evidence	cleaned_classroom_data.csv; processed_classroom_data.csv
Student-specific decision or observation	The current sample is useful for demonstrating the pipeline, but too small for broad operational conclusions.

4. Individual Work and Anti-Plagiarism Record

Individual evidence	Student entry
My unique technical/design decision	I use min_periods=3 so rolling statistics appear only after a complete three-minute window is available.
My own test/experiment	A boundary test checks that Temp_C = 24.0 and CO2_ppm = 1000 still produce comfort_flag = True.
Problem/bug I personally encountered and solved	After reading the CSV, Occupancy becomes float because the column contains a missing value. After forward fill, I convert it back to int so occupancy remains a whole-person count.
Exact code/repository/commit reference	main.py - clean_data(), add_rolling_statistics(), add_features(), calculate_statistics(), create_plots(); test_assignment2.py
One limitation of my implementation	The dataset contains only one room and twelve minutes of synthetic readings.
One improvement I propose	Extend the same pipeline to multiple rooms and longer streams, then compare room-level trends and alerts.

5. Evidence and Submission Checklist

- ☒ Technical report completed
- ☒ Source code submitted
- ☒ Required tables and calculations completed
- ☒ Required plots included
- ☒ Execution logs and reproducible calculation evidence included
- ☒ README and run instructions included

- ☒ Data files included
- ☒ Analytical results included
- ☐ Oral defense / demonstration completed

6. Student Declaration of Originality

I confirm that this submission represents my own work. I have not copied another student's source code, calculations, tables, architecture, plots, screenshots, logs, results, or written explanations. Any external material or reused code has been identified. I understand that the instructor may request a live demonstration, code modification, reproduction of calculations, or explanation of any submitted component.

Role	Name	Signature	Date
Student / Executor	Amankos Danial Dauletuly		21.09.2026

Appendix - Reproducible Evidence

The following artifacts are included in the submission folder:

evidence/main_output.txt - cleaning, rolling, feature, analysis, and interpretation output

evidence/pytest_output.txt - automated test result

evidence/calculation_check.json - selected numeric checks

cleaned_classroom_data.csv - cleaned source fields

processed_classroom_data.csv - rolling statistics and engineered features

plots/rolling_temperature_co2.png - required rolling plot

plots/occupancy_vs_power.png - required occupancy-power plot